

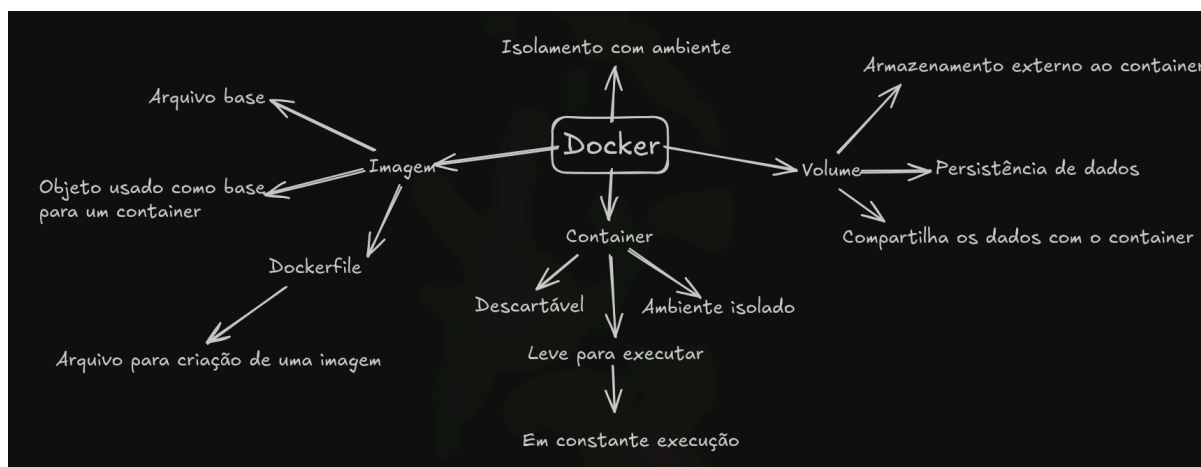
Relatório 18 - Docker e Containers para Aplicações (III)

Felipe Fonseca

1. Introdução

Nesse Card foi trabalhado os comandos básicos do docker, além de entender os principais objetos da aplicação, sendo elas, containers, imagens e volumes. Durante o card, foquei mais na parte teórica do que em dizer os comandos em si que foram mostrados, ao invés disso, todos os comandos que utilizei como prática estarão em um txt no github.

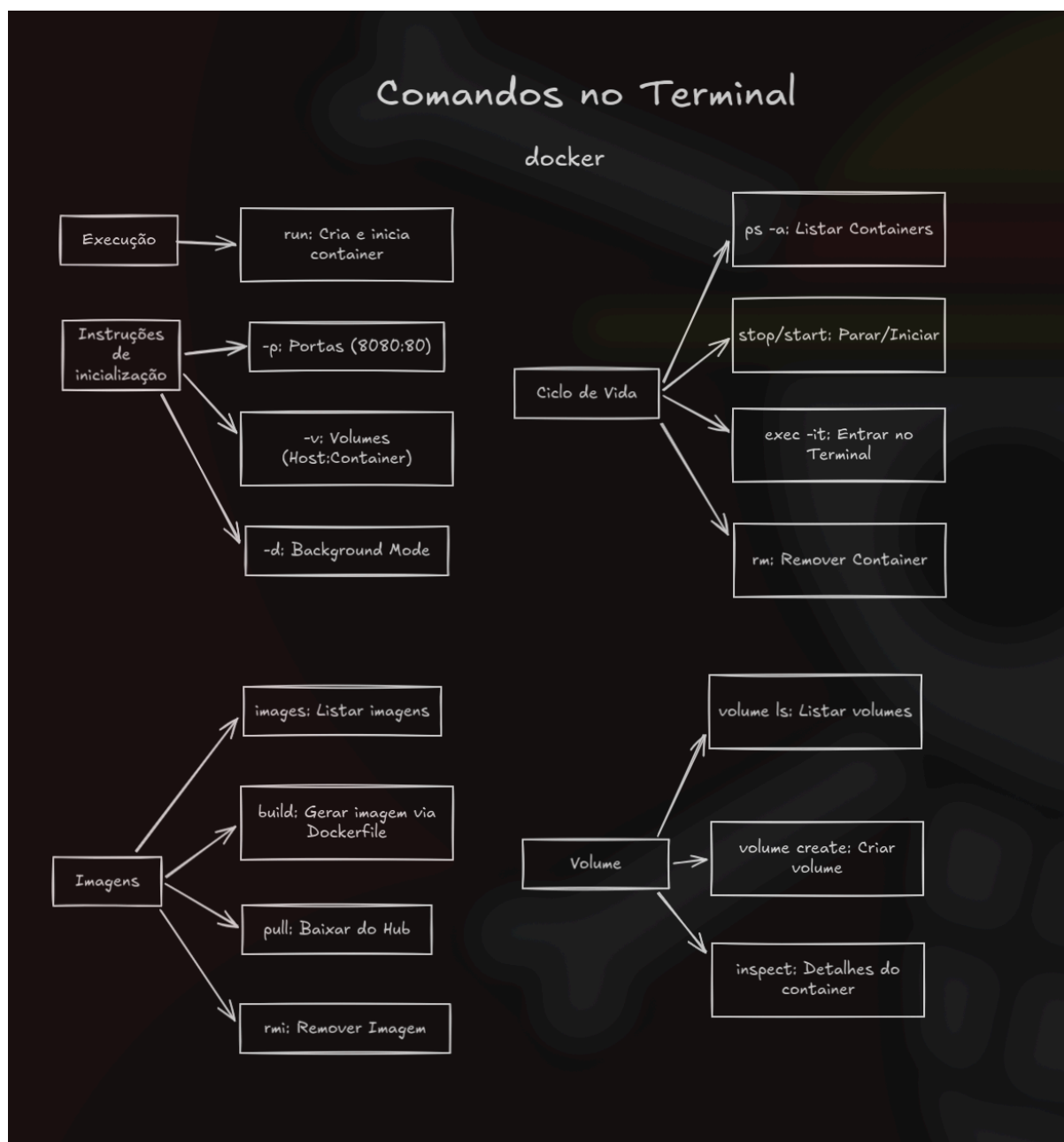
2. Desenvolvimento



Nesse card foi apresentado um tutorial sobre o docker. Docker é uma plataforma de código aberto que é utilizado para gerenciar containers, facilitando o controle para rodar softwares em diferentes plataformas. A ideia do docker é criar um ambiente descartável, onde podemos realizar qualquer teste sem alterar o sistema original, além de rodar aplicações sem alteração externa.

Utilizando o docker cli, ou seja, interface por linha de comando através do terminal, nós manipulamos 3 objetos no Docker, sendo eles, Containers, Imagens e Volumes.

Alguns dos principais comandos utilizados no docker:



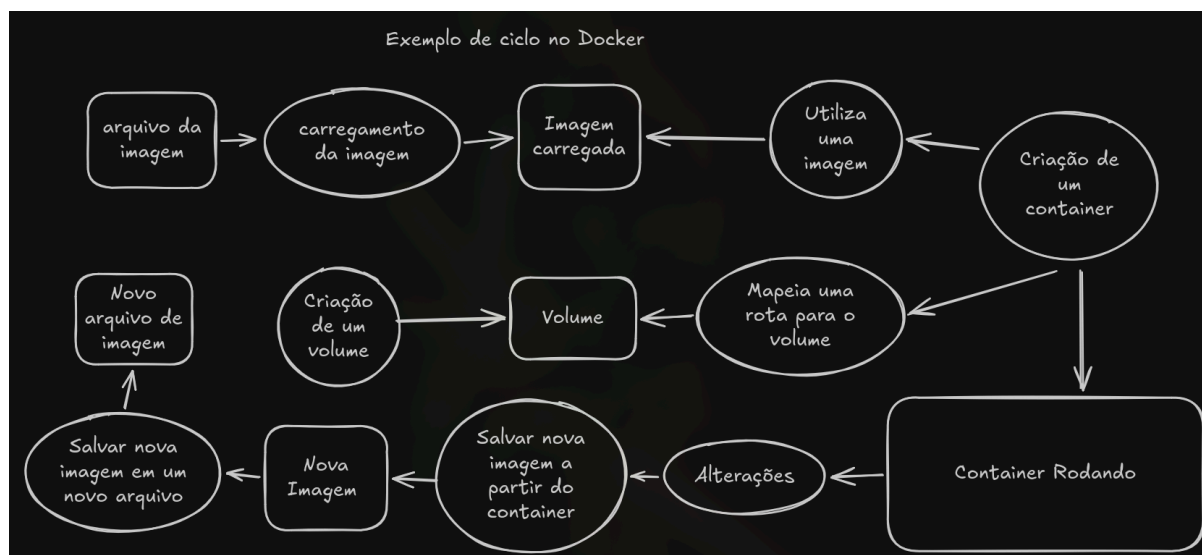
Começando com os containers, eles são o principal objeto do docker, e são eles os responsáveis pela ideia principal do projeto. Containers são exatamente os objetos responsáveis pelo ambiente que eu citei anteriormente, eles são ambientes descartáveis, isolados, que podem ser utilizados para testar aplicações em um ambiente controlado, são uma ferramenta poderosa para eliminar o problema do "no meu computador roda". Os containers são bem controláveis, eles podem ser pausados, removidos, copiados, também são fáceis de se criar novos containers ou salvar coisas que foram feitas nos containers e replicá-las em um container novo.

Um dos principais exemplos que chamaram a atenção nesse vídeo, foi justamente o teste de entrar dentro de um container que continha uma versão simples do Linux, na qual deletamos todo o sistema de arquivos.

O segundo objeto que foi apresentado foi a imagem, que é uma imagem gerada a partir de um container, que também pode ser utilizado para gerar um novo container. Alguns atributos das imagens são: não pode ser alterada e é salva normalmente em um arquivo tar. Para criar um container com uma imagem é utilizado o comando `docker commit`. Além disso, foi apresentado o `dockerfile`, que é um arquivo no qual podemos escrever uma sequência de comandos para criar uma imagem do docker personalizada, contendo bibliotecas, pastas e até arquivos locais incluídos, caso a gente queira.

O terceiro objeto que foi citado é o volume, que é uma pasta de dados "persistente", que fica fora do container, mas pode ser acessada por ele. Ou seja, é uma pasta que serve como transferência de arquivo de dentro do container para fora, e vice-versa, ao mesmo tempo que serve para persistência, ou seja, mesmo que o container seja deletado, a pasta se mantém.

Abaixo, um exemplo de ciclo de vida de um container do Docker:



Uma outra parte interessante que foi apresentada foi o mapeamento de portas, ou rotas. Basicamente, é possível mapear uma rota no container para acessá-lo por outro dispositivo que esteja na mesma rede. No exemplo feito com o vídeo, utilizamos um container rodando o servidor `nginx`, e mapeamos a rota, dessa forma, foi possível acessá-lo pelo navegador do sistema principal, por fora do docker.

Para não poluir o relatório com prints dos comandos que eu utilizei no terminal, irei colocar todos os comandos, assim como a saída inteira do terminal em arquivos `txt` no Github, juntamente do `Dockerfile`.

Basicamente eu fiz um ciclo de vida simples no docker, criei uma imagem a partir do `dockerfile`, depois criei um novo container a partir da imagem, fiz algumas alterações, testei se o volume criado no `dockerfile` estava funcionando, depois

commitei o novo container para uma nova imagem, e por fim salvei a imagem em um novo arquivo tar.

3. Conclusão

Este card serviu muito bem para entender os principais conceitos do Docker, assim como entender na prática como utilizar seus principais comandos, para criação de containers, imagens e volumes, além do mapeamento de portas, instruções etc. O Docker se mostrou muito útil pelo fato de ser descartável, ou seja, estando em um ambiente isolado e sendo descartável, é a aplicação perfeita para fazer qualquer tipo de testes e rodar aplicações diversas no mesmo sistema sem interferir uma na outra.

4. Referências

Vídeos do Card;

Resumo sobre o docker: <https://www.ibm.com/br-pt/think/topics/docker>;

Documentação do docker: <https://docs.docker.com/reference/cli/docker/container/run/>.